

## Isaac Belaich

[isaacbelaich@outlook.com](mailto:isaacbelaich@outlook.com)

[www.linkedin.com/in/isaac-belaich-a10752222](https://www.linkedin.com/in/isaac-belaich-a10752222)

<https://github.com/TheNewtonShow>

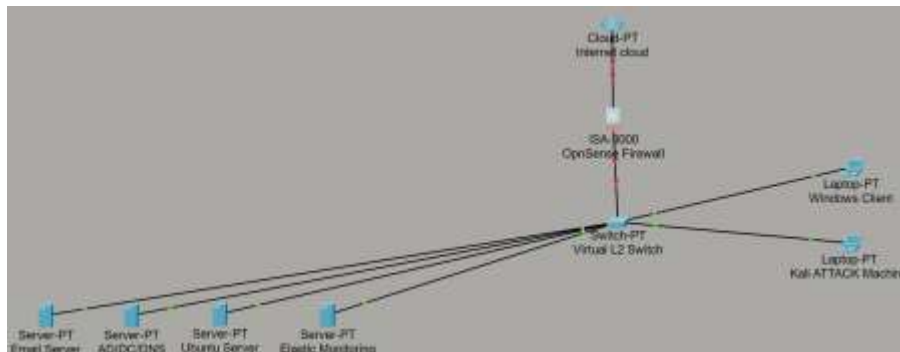
## Professional Summary

Dedicated and hands-on Cyber Security professional with practical experience in designing, building, and securing enterprise-level IT infrastructure. Skilled in virtualization, network administration, offensive penetration testing (Red Teaming), and defensive monitoring (Blue Teaming). Proven ability to overhaul vulnerable networks and implement hardened, centralized security architectures.

---

## Enterprise building Security lab

### Topology



**Doelstelling:** Het ontwerpen en bouwen van een realistisch, gelaagd en gevirtualiseerd bedrijfsnetwerk om een volledige IT-infrastructuur-overhaul te simuleren voor het fictieve middelgrote bedrijf "CyberEx". Het project richtte zich op de migratie van een kwetsbare, ongeorganiseerde werkgroep naar een hoogbeveiligde, gecentraliseerde en actief gemonitorde domeinomgeving.

- **Virtualization:** VMWare (Hypervisor), Virtual Networks (NAT & Internal Virtual Switches).
- **Networking & Security:** OPNsense (Virtual Firewall), pfSense concepts, VLANs, Link Aggregation (LAG), Subnetting.
- **Systems Administration:** Windows Server (Active Directory, DNS, Centralized File Shares), Windows 10 Clients, Ubuntu Server.
- **Offensive Security (Red Team):** Kali Linux, Penetration Testing Tools, Network Exploitation.

- **Defensive Security (Blue Team):** Elasticsearch Stack (SIEM/Log Monitoring), Sensei (Next-Gen Firewall Extensions), System Hardening.

## Key Technical Milestones & Achievements

### 1. Network Architecture & Virtualization

- **Architectuur:** Vanaf de basis een volledige virtuele netwerktopologie ontworpen en opgebouwd binnen VMware.
- **Segmentatie:** Het netwerk opgedeeld in strikt gescheiden zones, waarbij NAT is gebruikt voor externe WAN-toegang en interne virtuele switches voor beveiligde LAN-communicatie.
- **Optimalisatie:** Dynamisch groeiende virtuele schijven toegewezen en de prestaties van virtuele machines geoptimaliseerd door middel van guest add-ons en extensies.**Table:**

| Device Name              | Role / Description                  | Prefix | IP Address     | Subnet mask   | Allocation Type |
|--------------------------|-------------------------------------|--------|----------------|---------------|-----------------|
| <b>OPNsense</b>          | Next-Gen Firewall / Default Gateway | /24    | 10.200.200.254 | 255.255.255.0 | Static          |
| <b>Virtual L2 Switch</b> | Internal Network Switch             | /24    | 10.200.200.0   | 255.255.255.0 | /               |
| <b>Windows Server</b>    | AD / DC / DNS Server                | /24    | 10.200.200.10  | 255.255.255.0 | Static          |
| <b>Email Server</b>      | Internal Mail Services              | /24    | 10.200.200.11  | 255.255.255.0 | Static          |
| <b>Ubuntu Server</b>     | Web / Utility Services              | /24    | 10.200.200.12  | 255.255.255.0 | Static          |
| <b>Elastic Server</b>    | SIEM / Log Monitoring               | /24    | 10.200.200.13  | 255.255.255.0 | Static          |
| <b>Windows 10</b>        | Standard User Client                | /24    | 10.200.200.50  | 255.255.255.0 | DHCP / Static   |
| <b>Kali Linux</b>        | Red Team Attack Machine             | /24    | 10.200.200.99  | 255.255.255.0 | Static          |

### 2. Perimeter Security & Firewall Implementation

- Deployed and configured an **OPNsense Firewall** to act as the primary gateway between the WAN and internal LAN.
- Manually configured network interfaces via the command line, assigning static IP addresses (e.g., 10.200.200.x /24) and routing protocols.

- Configured web graphical user interface (GUI) access over secure HTTPS protocols for remote firewall administration .
- Prepared the firewall for Next-Generation Intrusion Detection and Prevention (IDS/IPS) add-ons to filter malicious web traffic.

### 3. Enterprise Systems Administration



- Overhauled a peer-to-peer workgroup by deploying a Windows Server to act as a primary Domain Controller.
- Implemented Active Directory for centralized user management, centralized file storage, and DNS resolution, fixing previous lack of administrative control .
- Deployed Ubuntu Linux servers to host web services and perform administrative testing.

### 4. Red Team / Penetration Testing

- Integrated a Kali Linux attack machine directly into the internal network to simulate insider threats and unauthorized BYOD devices.
- Conducted hands-on penetration testing and executed simulated hacks against the unhardened Windows 10 clients and servers to identify network vulnerabilities.

### 5. Blue Team / Threat Monitoring & Defense

- Built and integrated an **Elasticsearch Stack** to act as a centralized logging and monitoring system .

- Created visual dashboards to aggregate system logs, monitor for indicators of compromise (IoCs), and defend against modern threats like ransomware.
  - Applied strict system hardening techniques across all Windows and Linux devices.
- 

## **Chapter 1: Transparent Proxy & Web Filtering Configuration**

**Objective:** To enforce corporate internet usage policies and optimize network traffic by deploying a transparent web proxy and content filter. The primary business goal was to restrict employee access to time-wasting sites (like social media) without requiring manual proxy configurations on every client machine.

### **Tools & Technologies Demonstrated:**

- **Firewall/Routing:** OPNsense, Port Forwarding (NAT), Firewall Rule Prioritization.
- **Proxy & Filtering:** Squid Cache (Proxy), SquidGuard (Web Filtering), Transparent SSL Proxying (HTTPS Interception).
- **Cryptography & Trust:** Self-Signed Certificate Authorities (CA), Certificate Deployment, SSL Bumping.
- **Access Control:** Domain Blacklists (University of Toulouse ACLs).

### **Key Technical Milestones & Achievements:**

#### **1. Certificate Authority Creation & SSL Decryption (Man-in-the-Middle)**

- Created an internal, self-signed Certificate Authority (CA) on the OPNsense firewall to facilitate SSL/HTTPS inspection.
- Exported and manually installed the root certificate onto the Windows client machine's "Trusted Root Certification Authorities" store, establishing a chain of trust between the client and the firewall proxy.
- Configured SSL "Bump Sites" to ensure that sensitive encrypted traffic (e.g., banking or financial websites) was bypassed and not decrypted by the proxy, maintaining user privacy and security compliance.

#### **2. Squid Proxy & Transparent Port Forwarding**

- Enabled the Squid Cache proxy service on OPNsense to cache static web content locally, optimizing network bandwidth and improving page load times for frequent requests.

- Configured the proxy to run in "Transparent Mode," meaning users are routed through the proxy automatically without needing to manually configure proxy settings in their web browsers.
- Created internal port forwarding (NAT) rules to forcefully redirect standard outbound HTTP (Port 80) and HTTPS (Port 443) traffic to the local proxy ports (3128 and 3129).

### 3. Enterprise Web Filtering & Access Control Lists (ACLs)

- Integrated external, actively maintained blacklist databases (provided by the University of Toulouse) into the SquidGuard web filtering engine.
- Parsed the downloaded Access Control Lists (ACLs) and successfully applied restrictions to the "Social Networks" category to meet the simulated company's administrative request.

### 4. Firewall Hardening against Proxy Evasion

- Engineered strict egress firewall rules on the LAN interface to explicitly block all outbound HTTP and HTTPS traffic that attempted to bypass the proxy server.
  - Demonstrated an understanding of firewall rule hierarchy (first-match basis) by manually prioritizing the new block rules above standard "allow all" rules, while carefully avoiding locking out administrative access to the firewall GUI.
  - Successfully validated the deployment by attempting to access a blocked social media site (Facebook) from the client machine, confirming that the traffic was intercepted and a customized "Access Denied" error was displayed.
- 

## Chapter 2: Next-Generation Firewall Deployment & Network Segmentation

**Objective:** To establish a secure perimeter and central routing hub for the enterprise lab by deploying an OPNsense firewall appliance. The primary goal was to successfully bridge the external internet (WAN) with an isolated, secure internal corporate network (LAN) using virtualization technologies.

### Tools & Technologies Demonstrated:

- **Firewall/Routing:** OPNsense (FreeBSD-based), pfSense concepts, Command Line Interface (CLI) configuration.
- **Virtualization:** VMWare (Hypervisor), Virtual Machine Provisioning, Virtual Network Adapters (NAT & Internal Networks), Guest Additions.

- **Networking:** IP Addressing, Subnetting, Interface Mapping, Static Routing.
- **Utilities:** Bzip2 (Decompression).

## Key Technical Milestones & Achievements:

### 1. Virtual Appliance Provisioning & OS Installation

- Downloaded and decompressed the OPNsense ISO media using command-line utilities.
- Provisioned a custom FreeBSD 64-bit Virtual Machine within VMWare, carefully allocating compute resources (vCPUs, RAM) and a dynamically expanding virtual disk to optimize host machine performance.

```
PS C:\Program Files (x86)\GnuWin32\bin> dir

Directory: C:\Program Files (x86)\GnuWin32\bin

Mode                LastWriteTime         Length Name
----                -
d----- 2000/03/20    21:12          384000 bzutil2.exe
d----- 2000/03/20    21:12          384000 bzcat.exe
d----- 2000/03/20    21:14           2128 bzcmp
d----- 2000/03/20    21:14           2128 bzdiff
d----- 2000/03/20    21:14           1632 bzegrep
d----- 2000/03/20    21:14           1632 bzfgrep
d----- 2000/03/20    21:14           1632 bzgrep
d----- 2000/03/20    21:12          94120 bzlib2.dll
d----- 2000/03/20    21:12          384000 bzlib2.exe
d----- 2000/03/20    21:12          19456 bzlib2pconv.exe
d----- 2000/03/20    21:14           1250 bzlex
d----- 2000/03/20    21:14           1250 bzmore

PS C:\Program Files (x86)\GnuWin32\bin> .\bzutil2.exe -help
bzutil2, a block-sorting file compressor.  Version 1.0.5, 10-Dec-2007.

usage: bzutil2.exe [-flags and input files in any order]

  -h --help                print this message
  -d --decompress          force decompression
  -c --compress            force compression
  -k --keep                keep (don't delete) input files
  -f --force               overwrite existing output files
  -t --test                test compressed file integrity
  -o --stdout              output to standard out
  -q --quiet               suppress noncritical error messages
  -v --verbose             be verbose (s and -v gives more)
  -l --license             display software license & license
  -V --version             display software version & license
  -m --small               use less memory (at most 250MB)
  -l .. -b                set block size to 100k .. 900k
  --fast                  alias for -3
  --best                  alias for -9

If invoked as 'bzcat', default action is to decompress.
as 'bzdiff', default action is to decompress.
as 'bzcat', default action is to decompress to stdout.

If no file names are given, bzutil2 compresses or decompresses
from standard input to standard output.  You can combine
```

- Booted the live environment and successfully executed the bare-metal installation of the OPNsense operating system onto a local Unix File System (UFS) partition.



- Established initial local administrative credentials and secured the root account.

## 2. Virtual Network Architecture & Interface Mapping

- Architected a dual-homed firewall configuration using VMWare's advanced network adapter settings.
- Mapped the first virtual interface (WAN) to a Network Address Translation (NAT) engine, allowing the firewall to securely pull an internet connection from the host machine.
- Mapped the second virtual interface (LAN) to an isolated "Internal Network," effectively creating a virtual Layer 2 switch strictly dedicated to the enterprise lab environment.

## 3. Command-Line Network Configuration

- Utilized the OPNsense command-line console to physically assign the hypervisor's virtual network interface cards (em0 and em1) to their designated WAN and LAN roles.
- Engineered the internal routing by assigning a static, Class C IP address (10.200.200.254/24) to the LAN interface, establishing it as the default gateway for all future internal servers and clients.

```
Setting up routes...done.
Starting Unbound DNS...done.
Setting up gateway monitors...done.
Configuring Firewall.....done.
Starting web GUI...done.

You can now access the web GUI by opening
the following URL in your web browser:

    http://10.200.200.254

*** OPNsense.localdomain: OPNsense 22.1 (amd64/OpenSSL) ***

LAN (em1)    -> v4: 10.200.200.254/24
WAN (em0)    -> v4/DHCP4: 10.0.2.15/24

8) Logout                7) Ping host
1) Assign interfaces      8) Shell
2) Set interface IP address 9) pftop
3) Reset the root password 10) Firewall log
4) Reset to factory defaults 11) Reload all services
5) Power off system        12) Update from console
6) Reboot system           13) Restore a backup

Enter an option: █
```

- Intentionally bypassed OPNsense's default DHCP server setup to allow for future deployment of a dedicated Windows Server Domain Controller for enterprise-grade IP management.

#### 4. Web GUI Initialization & System Optimization

- Deployed a Kali Linux node onto the internal virtual network and successfully tested ICMP (ping) connectivity to the newly established firewall gateway.
- Accessed the OPNsense Web Graphical User Interface (GUI) to verify system status, routing tables, and interface health.
- Updated the firewall's firmware repository and installed VMWare Guest Additions via the OPNsense plugin manager to ensure seamless integration and optimized performance between the hypervisor and the firewall OS.

### Chapter 3: Intrusion Detection/Prevention System (IDS/IPS) Deployment & Threat Hunting

**Objective:** To enhance the perimeter security of the virtual enterprise network by deploying and configuring Suricata-based Intrusion Detection and Prevention Systems (IDS/IPS) on the OPNsense firewall. The goal was to actively monitor LAN traffic for malicious patterns and successfully detect simulated reconnaissance attacks.

#### Tools & Technologies Demonstrated: \* Defensive (Blue Team):

OPNsense IDS/IPS (Suricata), Hyperscan algorithm, Custom Rule Writing, Log Analysis.

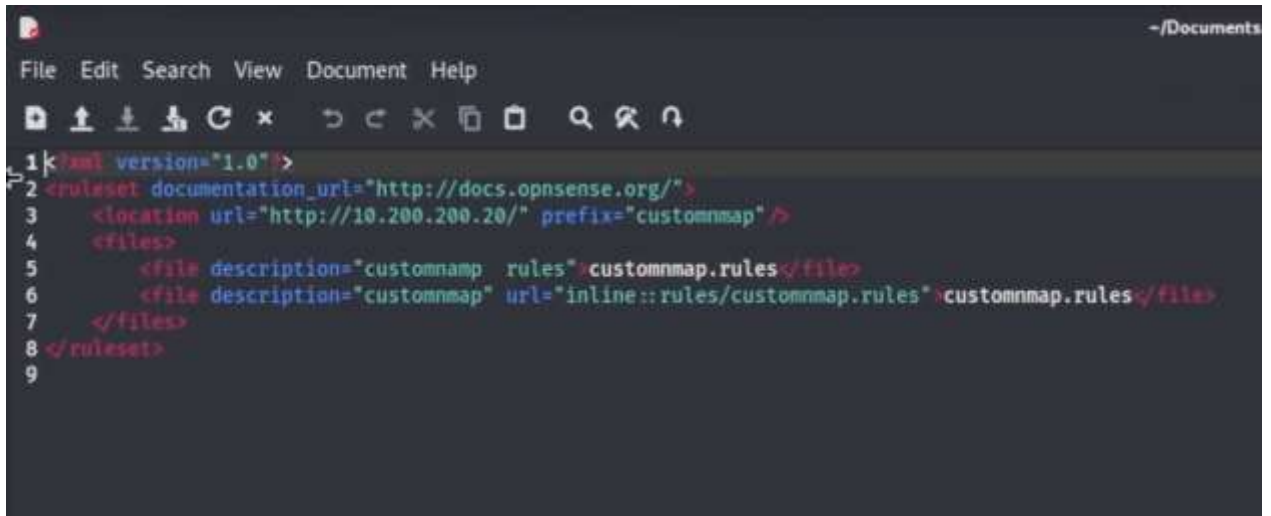
- **Offensive (Red Team):** Kali Linux, Nmap (Stealth SYN Scanning).
- **Administration:** SSH, SFTP (FileZilla), Python3 HTTP Server, XML configuration.

#### Key Technical Milestones & Achievements:



- ## 1. IDS/IPS Engine Configuration & Optimization

## Portfolio Cybersecurity



A screenshot of a text editor window showing an XML configuration file. The window has a menu bar with 'File', 'Edit', 'Search', 'View', 'Document', and 'Help'. Below the menu bar is a toolbar with various icons. The XML code is as follows:

```
1<?xml version="1.0"?>
2<ruleset documentation_url="http://docs.opnsense.org/">
3  <location url="http://10.200.200.20/" prefix="customnmap">
4    <files>
5      <file description="customnmap rules">customnmap.rules</file>
6      <file description="customnmap url="inline::rules/customnmap.rules">customnmap.rules</file>
7    </files>
8  </ruleset>
9
```

- Configured the rule parameters to trigger an alert if 50 TCP SYN requests were detected within a 1-second threshold, classifying it as "Attempted Recon".



A screenshot of a terminal window showing an Nmap scan. The window has a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. The terminal output is as follows:

```
(kali@kali)-[~]
$ sudo nmap -sS -Pn -p-top-ports 500 10.200.200.254
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2022-02-17 14:51 EST
Nmap scan report for 10.200.200.254
Host is up (0.00052s latency).
Not shown: 497 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
MAC Address: 08:00:27:8D:13:2D (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 3.71 seconds

(kali@kali)-[~]
$
```

## Portfolio Cybersecurity

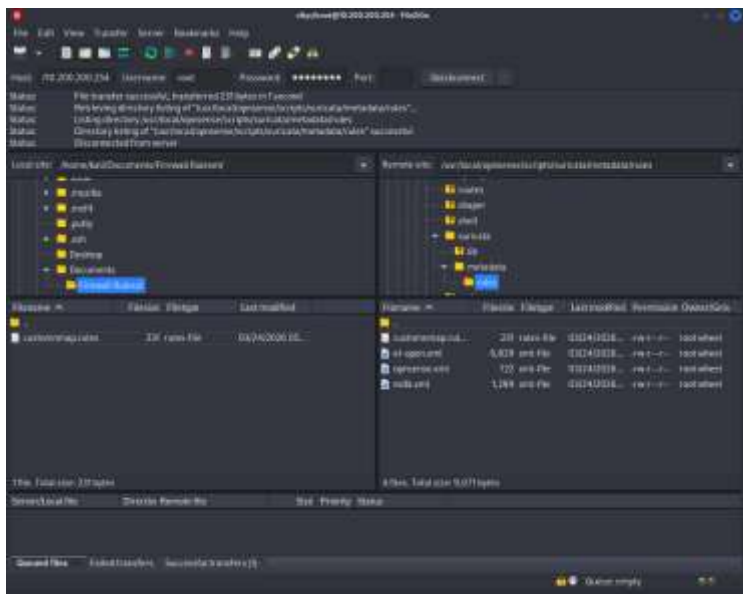
Services: Intrusion Detection: Administration

2022/02/17 09:54

| Timestamp                     | ID   | Action  | Interface | Source        | Port  | Destination    | Port  | Alert                                  | Info |
|-------------------------------|------|---------|-----------|---------------|-------|----------------|-------|----------------------------------------|------|
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 497   | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 52765 | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 165   | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 52775 | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 5153  | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 5     | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |
| 2022-02-17T12:51:32.842Z+0000 | 1234 | allowed | eth       | 10.200.200.20 | 55449 | 10.200.200.254 | 1009  | POSSIBLE NMAP SNIFFLEATH SCAN DETECTED |      |

### 3. Secure File Transfer & Dynamic Rule Ingestion

- Temporarily enabled secure shell (SSH/SFTP) access on the OPNsense firewall to allow for advanced administrative file management .
- Utilized FileZilla to securely transfer a custom .xml configuration file into the OPNsense root directory .

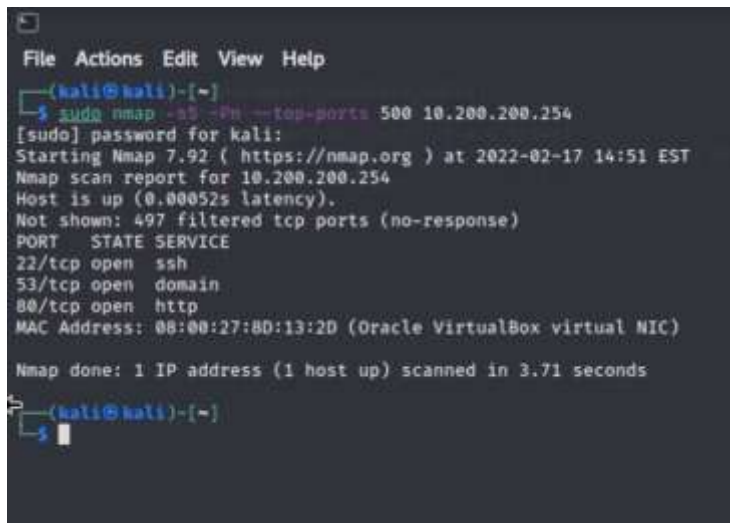


- Deployed a lightweight Python 3 HTTP server on the Kali Linux machine to host the custom .rules file, allowing the OPNsense firewall to dynamically query and ingest the threat intelligence over the network .

```
File Actions Edit View Help
(kali@kali) [~/Documents/suricata_rules]
python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

#### 4. Attack Simulation & Defense Validation

- Transitioned to the Red Team role to validate the defense mechanisms.
- Executed an Nmap Stealth SYN scan (sudo nmap -sS -Pn --top-ports 500 10.200.200.254) from the Kali attack machine against the firewall gateway.



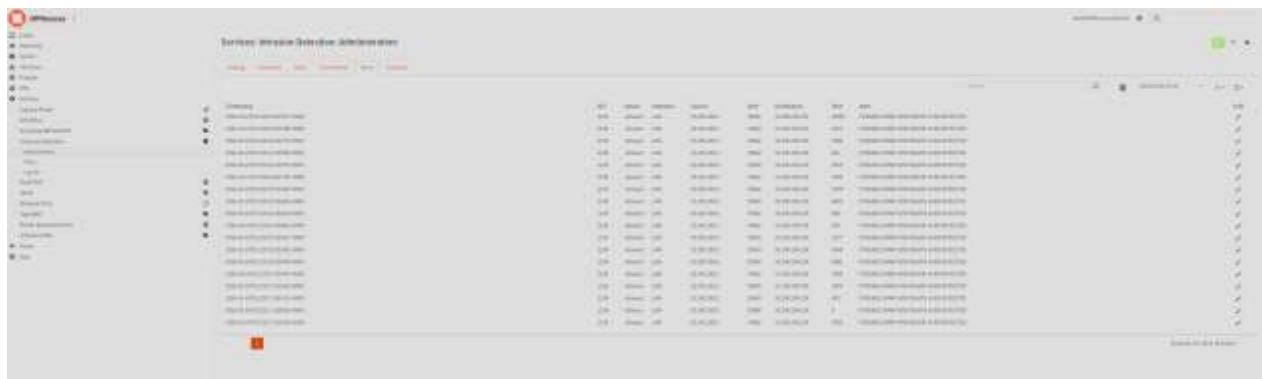
```

File Actions Edit View Help
(kali@kali)-[~]
└─$ sudo nmap -sS -Pn --top-ports 500 10.200.200.254
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2022-02-17 14:51 EST
Nmap scan report for 10.200.200.254
Host is up (0.00052s latency).
Not shown: 497 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
MAC Address: 08:00:27:8D:13:2D (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 3.71 seconds
(kali@kali)-[~]
└─$

```

- Transitioned back to the Blue Team role to analyze the OPNsense security dashboard, successfully verifying that the custom IDS rule caught the anomalous traffic, logged the attacker's source IP, and generated the expected "Possible Nmap Stealth Scan" alert.



| id  | Category | Event                       | Source         | Destination  | Port | Protocol | Time                | Alert                      |
|-----|----------|-----------------------------|----------------|--------------|------|----------|---------------------|----------------------------|
| 101 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 22   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 102 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 53   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 103 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 80   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 104 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 22   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 105 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 53   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 106 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 80   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 107 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 22   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 108 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 53   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 109 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 80   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 110 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 22   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 111 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 53   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 112 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 80   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 113 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 22   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 114 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 53   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |
| 115 | Network  | Network Intrusion Detection | 10.200.200.254 | 10.200.200.1 | 80   | TCP      | 2022-02-17 14:51:10 | Possible Nmap Stealth Scan |

#### Chapter 4: Transparent Proxy & Web Filtering Configuration

**Objective:** To enforce corporate internet usage policies and optimize network traffic by deploying a transparent web proxy and content filter. The primary business goal was to restrict employee access to time-wasting sites (like social media) without requiring manual proxy configurations on every client machine.

## Tools & Technologies Demonstrated:

- **Firewall/Routing:** OPNsense, Port Forwarding (NAT), Firewall Rule Prioritization.
- **Proxy & Filtering:** Squid Cache (Proxy), SquidGuard (Web Filtering), Transparent SSL Proxying (HTTPS Interception).
- **Cryptography & Trust:** Self-Signed Certificate Authorities (CA), Certificate Deployment, SSL Bumping.
- **Access Control:** Domain Blacklists (University of Toulouse ACLs).

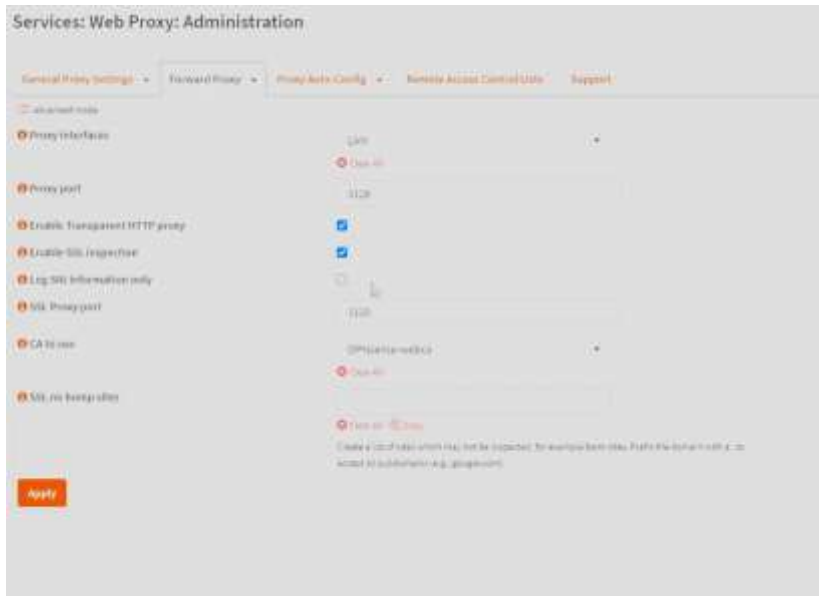
## Key Technical Milestones & Achievements:

### 1. Certificate Authority Creation & SSL Decryption (Man-in-the-Middle)

- Created an internal, self-signed Certificate Authority (CA) on the OPNsense firewall to facilitate SSL/HTTPS inspection.



- Exported and manually installed the root certificate onto the Windows client machine's "Trusted Root Certification Authorities" store, establishing a chain of trust between the client and the firewall proxy.



- Configured SSL "Bump Sites" to ensure that sensitive encrypted traffic (e.g., banking or financial websites) was bypassed and not decrypted by the proxy, maintaining user privacy and strict security compliance.

### 2. Squid Proxy & Transparent Port Forwarding

- Enabled the Squid Cache proxy service on OPNsense to cache static web content locally, optimizing network bandwidth and improving page load times for frequent requests.
- Configured the proxy to run in "Transparent Mode," meaning users are routed through the proxy automatically at the network level without needing to manually configure proxy settings in their individual web browsers.
- Created internal port forwarding (NAT) rules to forcefully redirect standard outbound HTTP (Port 80) and HTTPS (Port 443) traffic to the local proxy listener ports.

### 3. Enterprise Web Filtering & Access Control Lists (ACLs)

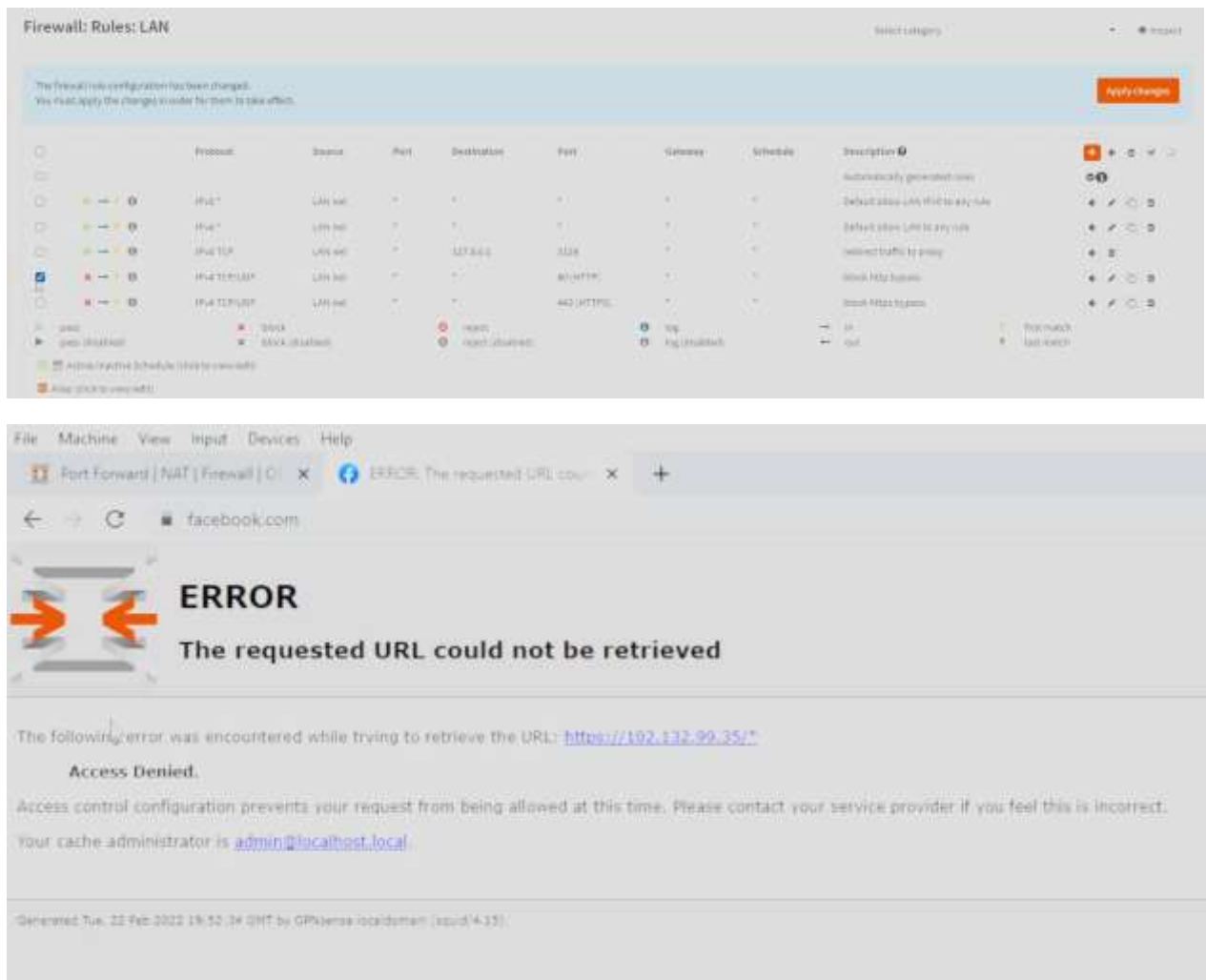
- Integrated external, actively maintained blacklist databases (provided by the University of Toulouse) directly into the SquidGuard web filtering engine.
- Parsed the downloaded Access Control Lists (ACLs) and successfully applied restrictions to the "Social Networks" category to meet the simulated company's administrative request.

### 4. Firewall Hardening against Proxy Evasion

- Engineered strict egress firewall rules on the LAN interface to explicitly block all outbound HTTP and HTTPS traffic that attempted to bypass the proxy server.

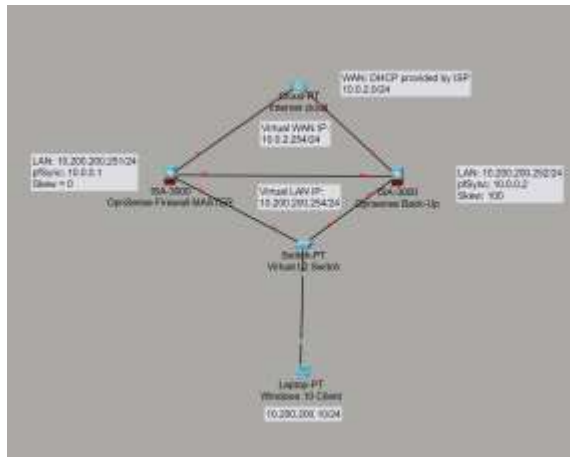
## Portfolio Cybersecurity

- Demonstrated a strong understanding of firewall rule hierarchy (first-match basis) by manually prioritizing the new block rules above standard "allow all" rules, while carefully avoiding locking out administrative access to the firewall GUI.
- Successfully validated the deployment by attempting to access a blocked social media site from the client machine, confirming that the traffic was seamlessly intercepted and a customized "Access Denied" error was displayed.



## Chapter 5: Enterprise High Availability (HA) & Redundancy Configuration

## Topology:



**Objective:** To eliminate single points of failure within the network perimeter by deploying a High Availability (HA) firewall cluster. The goal was to establish a redundant backup gateway capable of automatic, seamless failover to ensure continuous business operations if the primary firewall experiences a critical outage.

## Tools & Technologies Demonstrated:

- **Clustering & Redundancy:** High Availability (HA), CARP (Common Address Redundancy Protocol).
- **State & Config Synchronization:** pfSync (Stateful Failover), XML-RPC (Configuration Sync).
- **Networking:** Virtual IPs (VIPs), Hybrid Outbound NAT, Dedicated Sync Interfaces.
- **Virtualization:** VMWare Appliance Cloning, MAC Regeneration, NAT Networks (Layer 2 Simulation).

## Key Technical Milestones & Achievements:

### 1. Virtual Appliance Cloning & Network Preparation

- Successfully cloned the primary OPNsense Virtual Machine to act as the secondary/backup firewall, regenerating MAC addresses to prevent network conflicts.
- Migrated the WAN interfaces to a VMWare "NAT Network," effectively simulating a shared physical Layer 2 WAN switch where both firewalls could pull internet access and communicate directly.
- Provisioned and mapped a dedicated, isolated third network interface (NIC) strictly for synchronization traffic, ensuring sensitive heartbeat and state data remains secure from the LAN and WAN.



## 2. CARP & Virtual IP (VIP) Configuration

- Deployed CARP (Common Address Redundancy Protocol) to create floating Virtual IP (VIP) addresses shared between the master and backup firewalls on both the LAN (10.200.200.254) and WAN interfaces.
- Manipulated "Skew" weight values within the CARP groups to dictate network priority, configuring the primary firewall (Skew 0) to actively route traffic while forcing the secondary firewall (Skew 100) into a passive standby state.



## 3. State Replication (pfSync) & Configuration Sync (XML-RPC)

- Configured static IP addressing (10.0.0.1/24 and 10.0.0.2/24) on the dedicated sync interfaces and explicitly permitted synchronization protocols through the firewall rules.
- Enabled **pfSync** to continuously replicate active network connections, NAT tables, and session states from the master to the backup, ensuring that active user sessions would not be dropped during a failover event.
- Implemented **XML-RPC** synchronization on the master node to automatically push administrative configuration changes (rules, certificates, users) directly to the backup node, ensuring identical security postures.

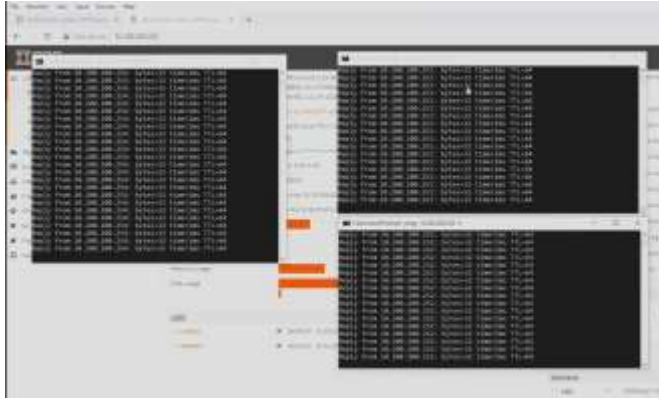
## 4. Outbound NAT Optimization

- Transitioned the firewall from automatic to Hybrid Outbound NAT configuration.
- Engineered a custom NAT rule dictating that all outbound traffic from the internal LAN must be translated to the shared WAN Virtual IP, rather than the firewall's physical interface IP. This ensured external routing and internet access would persist regardless of which firewall was physically handling the traffic.

## 5. Disaster Simulation & Failover Validation

- Established continuous ICMP monitoring streams against the master IP, backup IP, and shared Virtual Gateway IP.
- Simulated a catastrophic hardware/network failure by manually disconnecting the virtual network cables of the active master firewall.

- Successfully verified the failover mechanism: the backup firewall instantly detected the missing CARP heartbeat, promoted itself to the "Master" role, and resumed routing traffic with only a single dropped packet and zero noticeable disruption to simulated end-users.



### Chapter 6: Multi-WAN Failover & Load Balancing Implementation

**Objective:** To ensure continuous outbound internet connectivity and business operations by implementing a Multi-WAN architecture on the OPNsense firewall. The goal was to configure automatic failover to a backup Internet Service Provider (ISP) in the event of primary link degradation, as well as to demonstrate the ability to load-balance traffic across multiple ISPs.

#### Tools & Technologies Demonstrated:

- **Routing & Resilience:** Multi-WAN Networking, Gateway Groups, Gateway Tiering, Link Weighting.
- **Monitoring & Diagnostics:** Active Gateway Monitoring, ICMP Polling, Traceroute (traceroute -I).
- **Firewall/System:** OPNsense Default Gateway Switching, Policy-Based Routing (PBR), Interface-Specific DNS.
- **Virtualization:** VMWare Multi-NAT Networks (ISP Simulation).

#### Key Technical Milestones & Achievements:

##### 1. Multi-ISP Virtual Simulation & Interface Provisioning

- Engineered two distinct, isolated NAT networks within VMWare (10.0.2.0/24 and 10.0.3.0/24) to simulate two completely independent Internet Service Providers (WAN1 and WAN2).

- Provisioned a third virtual Network Interface Card (NIC) on the OPNsense firewall appliance and mapped it via the command line and web GUI to act as the secondary external WAN interface.



| Name              | Interface | Protocol | Priority | Gateway | Monitor IP | RTT    | RTT% | Loss | Status | Description                 |
|-------------------|-----------|----------|----------|---------|------------|--------|------|------|--------|-----------------------------|
| WAN_DHCP (active) | WAN       | IPv4     | 1        | 192.1   | 8.8.4.4    | 35.2ms | 2.4% | 0.0% | Active | Interface/WAN_DHCP Gateway  |
| WAN2_DHCP         | WAN2      | IPv4     | 254      | 192.1   | 8.8.8.8    | 35.4ms | 2.5% | 0.0% | Active | Interface/WAN2_DHCP Gateway |

## 2. Active Gateway Monitoring & Tiered Grouping

- Enabled continuous Gateway Monitoring on both WAN interfaces. Configured the firewall to actively poll specific external IP addresses (e.g., Google DNS 8.8.4.4 and 8.8.8.8) via ICMP to constantly evaluate link health, latency, and packet loss.
- Architected a **Gateway Group** to logically combine the two internet connections.
- Implemented a strict failover hierarchy by assigning the primary ISP to "Tier 1" and the backup ISP to "Tier 2", defining specific packet-loss thresholds that would trigger an automatic routing transition.

## 3. System DNS & Policy-Based Routing (PBR)

- Bound specific upstream DNS servers directly to their respective WAN interfaces to prevent DNS resolution failures during a partial network outage.
- Enabled "Default Gateway Switching" at the system level, allowing the firewall engine to dynamically rewrite its default routes on the fly if the Tier 1 gateway failed.
- Modified the core LAN firewall rules, explicitly changing the outbound gateway from the default WAN to the newly created Multi-WAN Gateway Group, ensuring all internal client traffic adhered to the failover policies.

## 4. Outage Simulation & Path Validation

## Portfolio Cybersecurity

- Conducted baseline traffic pathing using traceroute from the internal Kali client, confirming that packets successfully exited the network via the primary WAN1 gateway.
- Simulated a catastrophic ISP outage by manually disconnecting the virtual network cable associated with the WAN1 interface.
- Successfully validated the automated failover mechanism: the firewall instantly detected 100% packet loss on the primary monitor, demoted the WAN1 route, and seamlessly redirected the traceroute path through the secondary WAN2 gateway without requiring manual administrative intervention.



## Chapter 7: Next-Generation Firewall (NGFW) Upgrades & Deep Packet Inspection

**Objective:** To modernize the perimeter defenses by upgrading the traditional stateful OPNsense firewall into a Next-Generation Firewall (NGFW). The goal was to implement Layer 7 (Application Layer) visibility, Deep Packet Inspection (DPI), and real-time threat intelligence to identify and block advanced threats that evade standard port-based filtering.

### Tools & Technologies Demonstrated:

- **NGFW Engine:** Zenarmor (formerly OS Sensei) Plugin.
- **Traffic Analysis:** Deep Packet Inspection (DPI), Layer 7 Application Control.
- **Threat Intelligence:** Cloud Threat Intel Nodes, Web Categorization.
- **Logging & Analytics:** Local Elasticsearch Database Integration, Live Session Explorer.

### Key Technical Milestones & Achievements:

#### 1. NGFW Engine Provisioning & Database Integration

- Successfully added the Sunny Valley Networks repository and installed the Zenarmor (Sensei) engine directly onto the OPNsense appliance.
- Provisioned and integrated a local Elasticsearch database specifically to handle the high volume of logs and analytics generated by Deep Packet Inspection.

## Portfolio Cybersecurity

- Deployed the engine in "Routed Mode" (Layer 3) and bound it to the internal LAN interface to protect outbound user traffic.



## 2. Cloud Threat Intelligence & Security Policies

- Connected the firewall to European and US East Cloud Nodes to ingest real-time threat intelligence and web categorization feeds.
- Configured core security policies to leverage this intelligence, enabling the firewall to automatically identify and block traffic destined for known malware distribution points, phishing servers, and spam sites.

[illegible]

### 3. Layer 7 Application & Web Control

- Transitioned from basic port-based blocking (Layer 4) to application-aware filtering (Layer 7).

## Portfolio Cybersecurity

- Demonstrated the ability to create granular application controls, allowing the network to block specific services (like individual gaming platforms, specific webmail clients, or ad-trackers) regardless of what port or protocol they attempt to use to communicate.



### 4. Advanced Network Visibility & Threat Hunting Analytics

- Utilized the Zenarmor reporting dashboards to gain enterprise-level visibility into network traffic patterns.
- Analyzed Live Session Explorers to monitor active connections going through the firewall in real-time, allowing for immediate administrative action (like manually blocking a session on the fly).

- Interpreted detailed network telemetry, including top remote hosts, egress user tracking, connections parsed by specific applications over time, and geographical destination heatmaps.

## **Chapter 8: Security Operations Center (SOC) Initialization & SIEM Deployment**

**Objective:** To establish the foundation of a simulated Security Operations Center (SOC) by deploying a centralized Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) platform. The goal was to gain deep visibility into internal network assets by actively gathering logs, analyzing anomalies, and monitoring endpoint integrity across multiple operating systems.

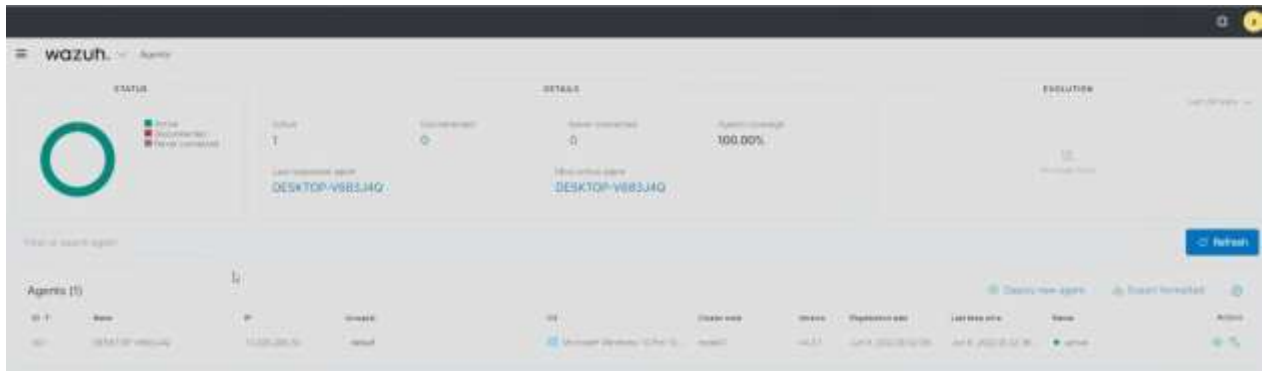
### **Tools & Technologies Demonstrated:**

- **SIEM / XDR Platform:** Wazuh (Open Source Security Platform).
- **Endpoint Management:** Wazuh Agent Deployment (Windows & Linux).
- **Systems Administration:** CentOS CLI, vi Editor, Network Interface Configuration, SSH.
- **Scripting & Automation:** PowerShell (Windows), Bash (Ubuntu).
- **Virtualization:** OVA Appliance Import, Virtual Disk Conversion (VMDK to VHD), Hyper-V / VMWare Provisioning.

### **Key Technical Milestones & Achievements:**

#### **1. SIEM Virtual Appliance Provisioning & OS Configuration**

- Downloaded the pre-built Wazuh Open Virtual Appliance (OVA) and demonstrated virtualization flexibility by utilizing command-line tools (VBoxManage) to convert the underlying virtual disk (.vmdk) into a format compatible with different hypervisors (.vhd).
- Provisioned the single-node Wazuh server cluster (allocating 4 vCPUs and 8GB RAM) to adequately handle data indexing, log processing, and dashboard analytics for the enterprise lab.
- Navigated the underlying CentOS command-line interface (CLI) using the vi text editor to modify the core network scripts (/etc/sysconfig/network-scripts/ifcfg-eth0). Successfully transitioned the SIEM from a dynamic DHCP lease to a strict static IP address (10.200.200.5) to ensure consistent agent reporting.



## 2. Cross-Platform XDR Agent Deployment (Windows)

- Accessed the secure Wazuh web dashboard and utilized the deployment wizard to configure an installation package tailored for the internal Windows 10 client.
- Executed the generated deployment script via an elevated PowerShell session, actively pointing the local XDR agent to the centralized Wazuh Manager IP.
- Verified the local Windows service initialization (`net start wazuh-svc`) and confirmed the endpoint successfully registered and populated within the SIEM dashboard.



## 3. Cross-Platform XDR Agent Deployment (Linux)

- Generated a secondary deployment script specifically formatted for Debian/Ubuntu architecture.
- Established an SSH connection from the Windows administrator machine into the internal Ubuntu 20.04 server.
- Executed the installation commands with root (`sudo`) privileges, successfully enrolling the Linux server into the SIEM and verifying cross-platform telemetry ingestion.

## 4. Centralized Telemetry & Dashboard Validation



- Successfully validated the SOC infrastructure by confirming that both the Windows workstation and the Linux server were actively checking in with the Wazuh Manager.
- Established the baseline for future threat hunting, file integrity monitoring (FIM), and vulnerability detection labs by ensuring the SIEM was actively receiving event logs from disparate network endpoints.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Users\jdoyle> Install-WazuhAgent -Path C:\ProgramData\Wazuh\bin -SourceUrl https://raw.githubusercontent.com/wazuh/wazuh-agent/master/wazuh-agent-4.4.1.exe -ServiceName WazuhAgent -LogPath C:\ProgramData\Wazuh\logs -Verbose
The Wazuh service is starting.
The Wazuh service has started successfully.
PS C:\Users\jdoyle>
```

## Chapter 9: Security Incident Response & Threat Intelligence Platform Deployment

**Objective:** To expand the enterprise Security Operations Center (SOC) capabilities by deploying a centralized Incident Response (IR) ecosystem. The goal was to establish a collaborative platform for analysts to track security incidents, analyze forensic observables, and ingest community-driven threat intelligence seamlessly.

### Tools & Technologies Demonstrated:

- **Incident Response & Intel:** TheHive (IR Platform), Cortex (Analysis Engine), MISP (Malware Information Sharing Platform).
- **Containerization:** Docker Engine, Docker Compose, Microservices Architecture.
- **Databases & Caching:** Cassandra (NoSQL), Elasticsearch (Analytics Engine), MySQL, Redis (In-Memory Data Store), MinIO (S3-compatible Object Storage).
- **Systems Administration:** Ubuntu Server 20.04, YAML Configuration, Persistent Volume Mapping.

### Key Technical Milestones & Achievements:

#### 1. Microservices Architecture & Containerization

- Transitioned from traditional monolithic virtual machine deployments to a modern, containerized microservices architecture using the Docker Engine installed on an Ubuntu 20.04 host.

```
PS C:\Windows\system32> ssh dockerlab@10.200.200.253
dockerlab@10.200.200.253's password:
welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-113-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Wed 15 Jun 2022 07:13:46 PM UTC

System load: 0.0                               Processes:                                     155
Usage of /: 23.6% of 28.67GB                    Users logged in:                               0
Memory usage: 2%                                IPv4 address for docker0: 172.17.0.1
Swap usage: 0%                                  IPv4 address for eth0: 10.200.200.253

27 updates can be applied immediately.
16 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Last login: Wed Jun 15 19:11:05 2022 from 10.200.200.30
dockerlab@dockerlab:~$ mkdir docker-compose
dockerlab@dockerlab:~$ ls

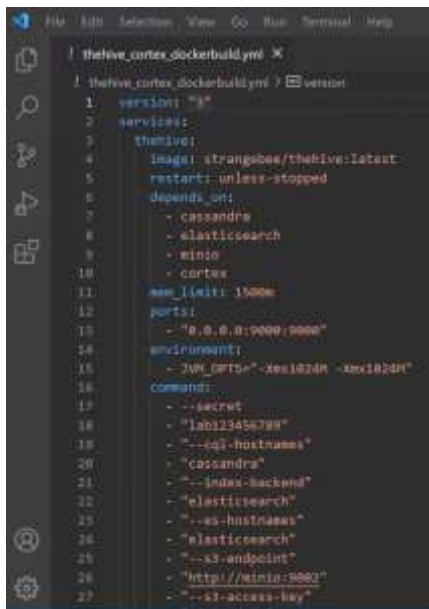
dockerlab@dockerlab:~$ cd docker-compose/
dockerlab@dockerlab:~/docker-compose$ nano
```

- Engineered a comprehensive docker-compose.yml configuration file to simultaneously define, link, and deploy eight distinct, interdependent application containers.

[illegible]

- Configured a custom isolated "bridged" Docker network (socknet) to secure internal backend communication between the containers while strictly mapping necessary ports

(e.g., 9000, 9001, 80, 443) to the host for administrative web access.

A screenshot of a text editor showing a Docker Compose file named 'thehive\_cortex\_dockerbuild.yml'. The file is written in YAML format and defines a service named 'thehive'. The service uses the 'strangebee/thehive:latest' image and has restarts configured as 'unless-stopped'. It depends on several other services: 'cassandra', 'elasticsearch', 'minio', and 'cortex'. The 'thehive' service is configured with environment variables for 'XMR\_OPTS' and 'XMR\_HOST', and a 'command' section with various flags and paths. The 'ports' section maps port 9000 on the host to port 9000 on the container. The 'networks' section is also present, defining a network named 'thehive' with a driver of 'bridge' and a range of '10.0.0.0/24'.

```
1 version: "3"
2 services:
3   thehive:
4     image: strangebee/thehive:latest
5     restart: unless-stopped
6     depends_on:
7       - cassandra
8       - elasticsearch
9       - minio
10      - cortex
11     ports:
12       - "9000:9000"
13     environment:
14       - "XMR_OPTS=-Xmx1024M -Xms1024M"
15     command:
16       - "--secret"
17       - "lab123456789"
18       - "--api-hostnames"
19       - "cassandra"
20       - "--index-backend"
21       - "elasticsearch"
22       - "--es-hostnames"
23       - "elasticsearch"
24       - "--s3-endpoint"
25       - "http://minio:9000"
26       - "--s3-access-key"
```

## 2. Persistent Storage & Database Provisioning

- Overcame Docker's ephemeral storage limitations by mapping external storage volumes directly to the host operating system, ensuring critical threat data and case files remain persistent even if a container crashes or restarts.
- Successfully orchestrated multiple backend databases simultaneously, utilizing Cassandra to handle high-performance NoSQL data, MySQL for MISP data retention, and Redis to cache frequently accessed intelligence in-memory.
- Deployed MinIO to act as a localized, private S3-compatible storage bucket for securing forensic files and case-related object data.

## 3. Application Deployment & Health Validation

- Executed the automated deployment (docker-compose up) to independently pull the latest images from Docker Hub, construct the internal network, and boot the interlinked ecosystem.
- Utilized Docker command-line utilities (docker ps) to validate the health, uptime, and port bindings of the entire container stack.
- Accessed and initialized the primary graphical user interfaces for TheHive, Cortex, and MISP, including forcing initial database schema updates and creating the master administrative accounts.

## 4. Incident Response Foundation

- Established the core infrastructure required to handle simulated cyber attacks. The deployed stack acts as a unified platform where TheHive creates the "Case," Cortex automatically analyzes attached observables (like malicious IPs or file hashes) against threat feeds, and MISP shares that malware intelligence with the broader security community.

### **Chapter 10: Incident Response Platform Integration & API Automation**

**Objective:** To fully integrate the containerized Incident Response ecosystem (TheHive, Cortex, and MISP) allowing for automated threat intelligence enrichment and streamlined case management. The goal was to eliminate manual intelligence gathering by configuring the platforms to communicate bidirectionally via RESTful APIs.

#### **Tools & Technologies Demonstrated:**

- **Incident Response & Intel:** TheHive (Case Management), Cortex (Analyzers), MISP (Threat Intelligence).
- **Identity & Access Management (IAM):** Multi-tenant Organizations, Role-Based Access Control (RBAC), Service Accounts.
- **API Integration:** RESTful API Key Generation and Authentication.
- **Configuration Management:** Docker Compose (docker-compose.yml), JSON/HOCON Configuration editing (application.conf), Persistent Volume Mapping.

#### **Key Technical Milestones & Achievements:**

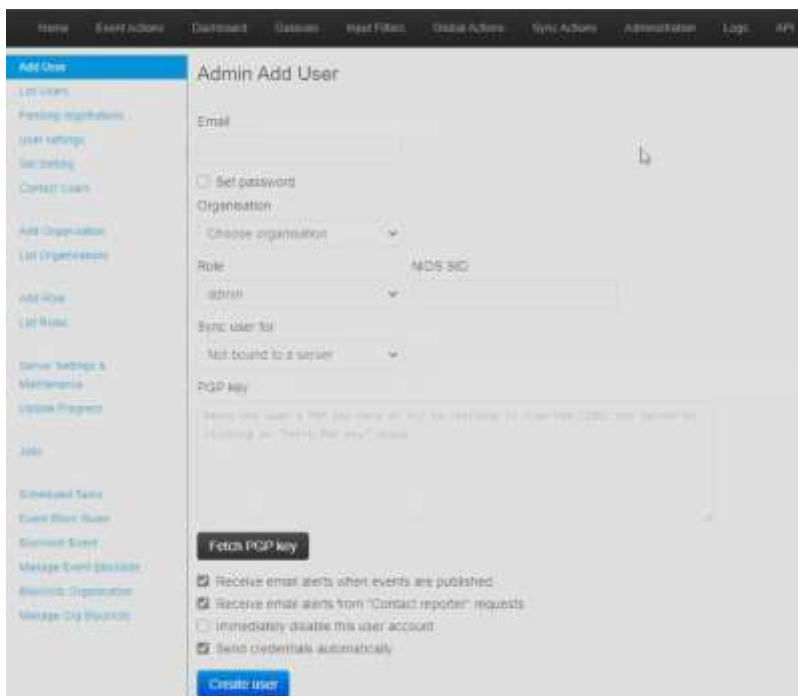
##### **1. Multi-Tenant Identity & Access Management (IAM)**

- Configured logical separation of assets by creating dedicated "Organizations" (e.g., Morgan Maxwell) within all three platforms: TheHive, Cortex, and MISP.
- Engineered Role-Based Access Control (RBAC) by provisioning specific user accounts (Org Admins, Analysts) and establishing strict password compliance policies across the ecosystem.
- Generated dedicated cryptographic Authentication/API keys within MISP and Cortex to allow for secure, programmatic, machine-to-machine communication without relying on standard user credentials.



## 2. Docker Configuration Management & Volume Mapping

- Interacted with the underlying Linux host to restructure the Docker deployment directory, explicitly creating configuration folders (/hive/conf) to hold application-specific parameter files.
- Wrote a custom application.conf file (utilizing HOCON syntax) for TheHive. Hardcoded the MISP container hostname and injected the previously generated MISP API key to explicitly define the synchronization intervals and authentication methods between the platforms.
- Edited the master docker-compose.yml file to inject the Cortex API key as an environmental variable and updated the persistent volume mappings to ensure TheHive container could accurately read the newly created application.conf file upon boot.



## 3. Infrastructure Re-Deployment & API Synchronization

- Gracefully tore down the running microservices stack (docker-compose down) and re-deployed the ecosystem (docker-compose up -d) to force the containers to ingest the new networking and API configurations.
- Troubleshot SSL/TLS certificate friction by configuring TheHive to securely bypass internal self-signed certificate warnings generated by the MISP container.

## 4. Ecosystem Validation & Dashboard Verification

- Logged into TheHive's platform management console to validate the API handshakes.
- Successfully verified that both Cortex and MISP registered as "Online" and fully synchronized within TheHive's dashboard (indicated by green health-check rings). This confirmed the automated pipeline was ready to accept security alerts from MISP and route forensic observables to Cortex for analysis.

